Assessment Set 1 — JSON (VARIANT) + Snowpark DataFrames Transformations

Field	Value
Assessment	Set 1 (Two tasks)
Target level	Medium (hands-on)
Total duration	30 minutes (15 + 15)
Environment	Snowsight: 1 SQL Worksheet + 1 Python Worksheet (Snowpark)
Submission	Screenshots only

Global Rules

- Use only the functions and features listed in each task (do not introduce extra Snowflake functions beyond what is allowed).
- Do not paste pre-written solutions from external sources; implement the required functionality yourself.
- Capture screenshots as you go. Screenshots are the submission artifact.
- All datasets are provided below as CREATE TABLE + INSERT statements (10 rows minimum).

1) Setup: Create Datasets

Run the following SQL in a SQL Worksheet. Re-running is safe (CREATE OR REPLACE).

A) Semi-Structured Dataset (VARIANT) — JSON_EVENTS

Used for JSON dot notation and bracket notation extraction. Contains nested objects + arrays, but you must NOT use extra functions such as FLATTEN in this assessment.

```
CREATE OR REPLACE TABLE JSON_EVENTS (
  EVENT_ID    INT,
  INGEST_TS   TIMESTAMP_NTZ,
  PAYLOAD     VARIANT
);

INSERT INTO JSON_EVENTS VALUES
(1,'2025-05-01 09:00:00',PARSE_JSON('{"user":{"id":"U001","name":"Asha"},"device":{"os":"Android","ap
(2,'2025-05-01 09:05:00',PARSE_JSON('{"user":{"id":"U002","name":"Ravi"},"device":{"os":"iOS","app_ve
(3,'2025-05-01 09:10:00',PARSE_JSON('{"user":{"id":"U003","name":"Mia"},"device":{"os":"Web","app_ver
(4,'2025-05-01 09:15:00',PARSE_JSON('{"user":{"id":"U001","name":"Asha"},"device":{"os":"Android","ap
(5,'2025-05-01 09:20:00',PARSE_JSON('{"user":{"id":"U004","name":"Noah"},"device":{"os":"Web","app_ve
(6,'2025-05-01 09:25:00',PARSE_JSON('{"user":{"id":"U005","name":"Emma"},"device":{"os":"Android","ap
(7,'2025-05-01 09:30:00',PARSE_JSON('{"user":{"id":"U006","name":"Oliver"},"device":{"os":"iOS","app_
(8,'2025-05-01 09:35:00',PARSE_JSON('{"user":{"id":"U003","name":"Mia"},"device":{"os":"Web","app_ver
(9,'2025-05-01 09:40:00',PARSE_JSON('{"user":{"id":"U007","name":"Sophia"},"device":{"os":"Android","
(10,'2025-05-01 09:45:00',PARSE_JSON('{"user":{"id":"U008","name":"Ethan"},"device":{"os":"Web","app_
```

B) Snowpark Tables — SALES_ORDERS and SALES_CUSTOMERS

Used for Snowpark DataFrame join + aggregates + filter + sort. Keep transformations in Snowpark (Python Worksheet).

```
CREATE OR REPLACE TABLE SALES_CUSTOMERS (
  CUSTOMER_ID  STRING,
  FULL_NAME    STRING,
  SEGMENT      STRING,
  COUNTRY      STRING,
  SIGNUP_DATE  DATE,
  STATUS       STRING
);

INSERT INTO SALES_CUSTOMERS VALUES
('C001','Asha Mehta','Consumer','IN','2025-01-05','ACTIVE'),
('C002','Ravi Nair','SMB','IN','2025-02-12','ACTIVE'),
('C003','Mia Johnson','Consumer','US','2025-01-18','ACTIVE'),
('C004','Noah Williams','Enterprise','US','2024-12-10','ACTIVE'),
('C005','Emma Brown','Consumer','UK','2025-03-02','ACTIVE'),
('C006','Oliver Smith','SMB','UK','2025-03-12','INACTIVE'),
('C007','Liam Garcia','Consumer','US','2025-04-01','ACTIVE'),
('C008','Sophia Martin','SMB','FR','2025-02-21','ACTIVE'),
('C009','Ethan Lee','Enterprise','SG','2025-01-28','ACTIVE'),
('C010','Isabella Kim','Consumer','SG','2025-03-20','ACTIVE');

CREATE OR REPLACE TABLE SALES_ORDERS (
  ORDER_ID      STRING,
  CUSTOMER_ID   STRING,
  ORDER_DATE    DATE,
  CHANNEL       STRING,
  ORDER_TOTAL   NUMBER(10,2),
  TAX_AMOUNT    NUMBER(10,2),
  ORDER_STATUS  STRING
);

INSERT INTO SALES_ORDERS VALUES
('O1001','C001','2025-04-01','Web',1499.00, 90.00,'DELIVERED'),
```

```
('O1002','C002','2025-04-02','Mobile', 799.00, 48.00,'DELIVERED'),
('O1003','C003','2025-04-02','Web', 120.00,  7.20,'CANCELLED'),
('O1004','C004','2025-04-03','Partner',5200.00,312.00,'DELIVERED'),
('O1005','C005','2025-04-03','Web', 980.00, 58.80,'DELIVERED'),
('O1006','C006','2025-04-04','Mobile',450.00, 27.00,'RETURNED'),
('O1007','C007','2025-04-05','Web',2100.00,126.00,'DELIVERED'),
('O1008','C008','2025-04-05','Web',1750.00,105.00,'DELIVERED'),
('O1009','C009','2025-04-06','Partner',12500.00,750.00,'DELIVERED'),
('O1010','C010','2025-04-06','Mobile',1300.00,78.00,'DELIVERED');
```

2) Assessment Tasks (Step-by-step Implementation Flow)

Task 1 — Semi-Structured JSON Handling (VARIANT)

Timebox: 15 minutes
Allowed functions/features (do not use extras): VARIANT column, PARSE_JSON(), Dot notation (payload:key.path), Bracket notation (payload['key']['path'])
Objective: Load JSON into a VARIANT column and extract nested fields using BOTH dot notation and bracket notation.

Implementation Flow (Trainee-friendly)

- In a SQL Worksheet, confirm JSON_EVENTS has 10 rows (simple COUNT is allowed only as validation of load).
- Write a SELECT that extracts: user id, user name, device os, event type, geo country, geo city.
- Use **dot notation** for at least 3 extracted fields.
- Use **bracket notation** for at least 3 extracted fields.
- Cast extracted fields into readable scalar types (STRING/BOOLEAN/NUMBER) so outputs are not VARIANT.
- Show a final output limited to 10 rows with clear column aliases.

Expected Output (what the evaluator should see)

- A tabular output with columns like USER_ID, USER_NAME, DEVICE_OS, EVENT_TYPE, COUNTRY, CITY.
- Values should be human-readable (not JSON blobs) and match inserted data (e.g., IN/Bengaluru for U001).
- Both notations are clearly present in your query (evaluator can see them).

Submission Screenshots (paste ALL)

- CREATE TABLE JSON_EVENTS and INSERT statements (or the executed SQL cell output).
- SELECT query showing dot notation usage.
- SELECT query showing bracket notation usage.
- Final query output with extracted columns (10 rows).

Common Mistakes (avoid)

- Do not use FLATTEN, OBJECT_KEYS, ARRAY functions, or any additional semi-structured helpers (not allowed).
- Do not keep extracted columns as raw VARIANT—cast to scalar types.
- Do not use a different dataset/table name.

Task 2 — Snowpark DataFrames: Join + Aggregates + Filter + Sort

Timebox: 15 minutes
Allowed functions/features (do not use extras): Snowpark DataFrames; join; aggregates: max, min, avg, sum; filter using total; sort numeric descending
Objective: Using Snowpark DataFrames, join orders and customers, compute summary metrics, filter by a total threshold, and sort the summary in descending order.

Implementation Flow (Trainee-friendly)

- Open a **Python Worksheet** (Snowpark) and set context (role/warehouse/database/schema).
- Load SALES_CUSTOMERS and SALES_ORDERS as DataFrames.
- Join them on CUSTOMER_ID to enrich orders with customer attributes.
- Filter rows to keep only ORDER_STATUS = 'DELIVERED' (deliveries only).
- Group by COUNTRY (or SEGMENT) and compute: SUM(ORDER_TOTAL), AVG(ORDER_TOTAL), MIN(ORDER_TOTAL), MAX(ORDER_TOTAL).
- Apply a filter on the aggregated **SUM(ORDER_TOTAL)** to keep only groups above a meaningful threshold.
- Sort the result by SUM(ORDER_TOTAL) in **descending** order and display output.

Expected Output (what the evaluator should see)

- Aggregated output with one row per group (e.g., COUNTRY) showing SUM/AVG/MIN/MAX.
- Groups with low totals are excluded by the total-based filter.
- Rows are sorted with the highest total at the top.

Submission Screenshots (paste ALL)

- Snowpark context configuration (role/warehouse/db/schema).
- DataFrame load + join logic screenshot.
- Aggregation logic screenshot showing max/min/avg/sum.
- Screenshot showing filter based on total (SUM) after aggregation.
- Final sorted output screenshot (descending totals).

Common Mistakes (avoid)

- Do not use extra transformations beyond join, aggregates, filter-on-total, and sort-desc.
- Do not compute aggregates in Python loops; use DataFrame aggregations only.
- Do not sort ascending; must be descending on the numeric total.